

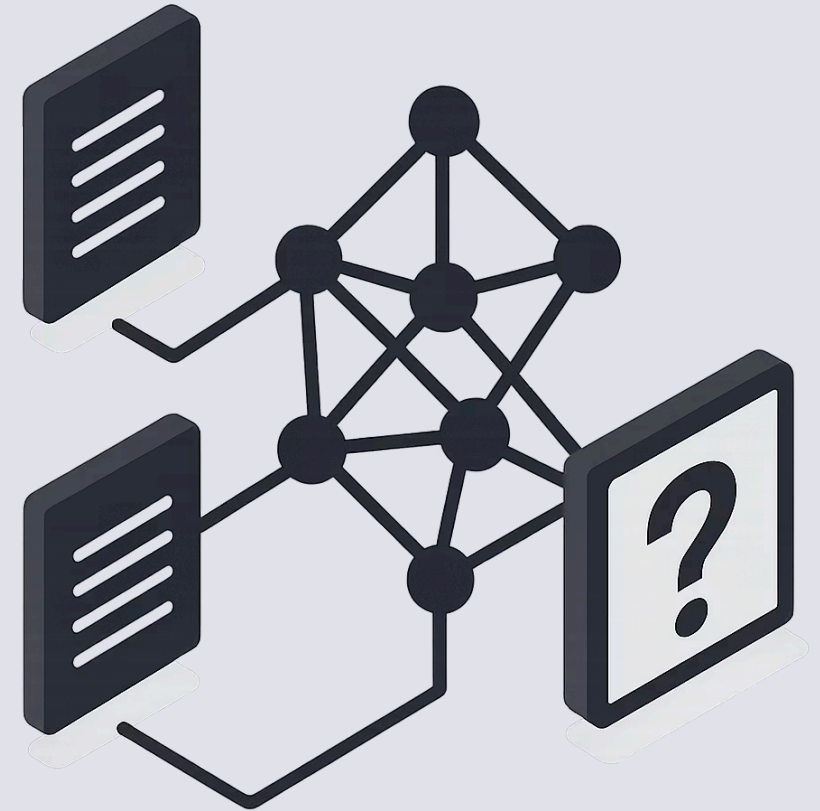
Retrieval-Augmented Generation (RAG)

Dr. A. V. Brahmane

Department of Computer Engineering

Sanjivani College of Engineering, Kopergaon

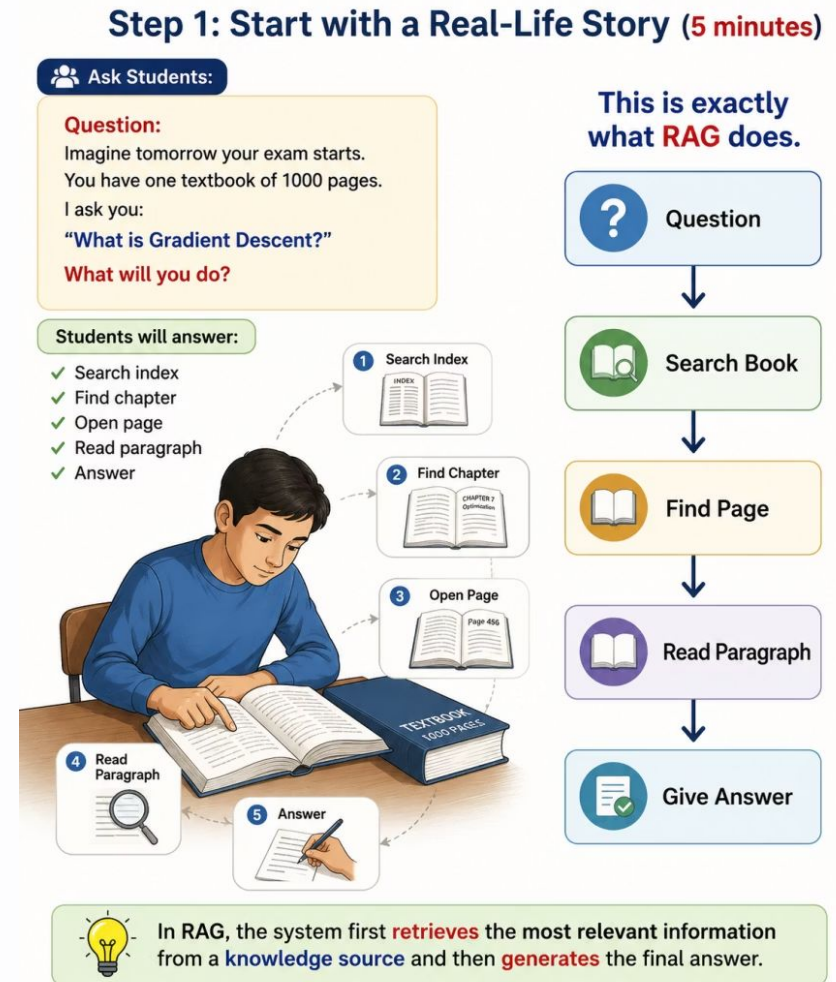
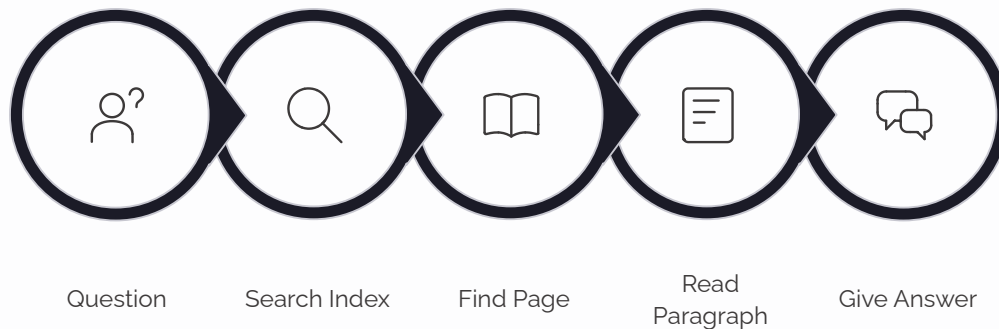
LECTURE SERIES · AI & NLP



Introduction to Retrieval-Augmented Generation (RAG)

Consider this scenario: your exam is tomorrow and you have a 1,000-page textbook. When asked *"What is Gradient Descent?"*, you don't read the entire book – you **search the index**, **find the chapter**, **open the page**, **read the paragraph**, and then answer.

This is **exactly** what RAG does. Instead of memorising everything upfront, RAG intelligently retrieves only the most relevant information before generating a response – making it faster, more accurate, and far more scalable than traditional AI approaches.



The Problem with ChatGPT Alone

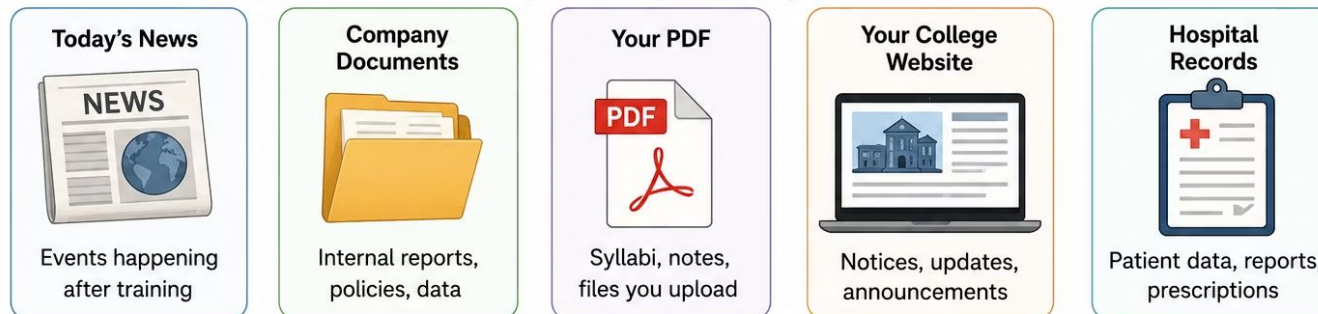
Step 2: Problem with ChatGPT Alone



Explain:
LLMs only know what they learned during training.



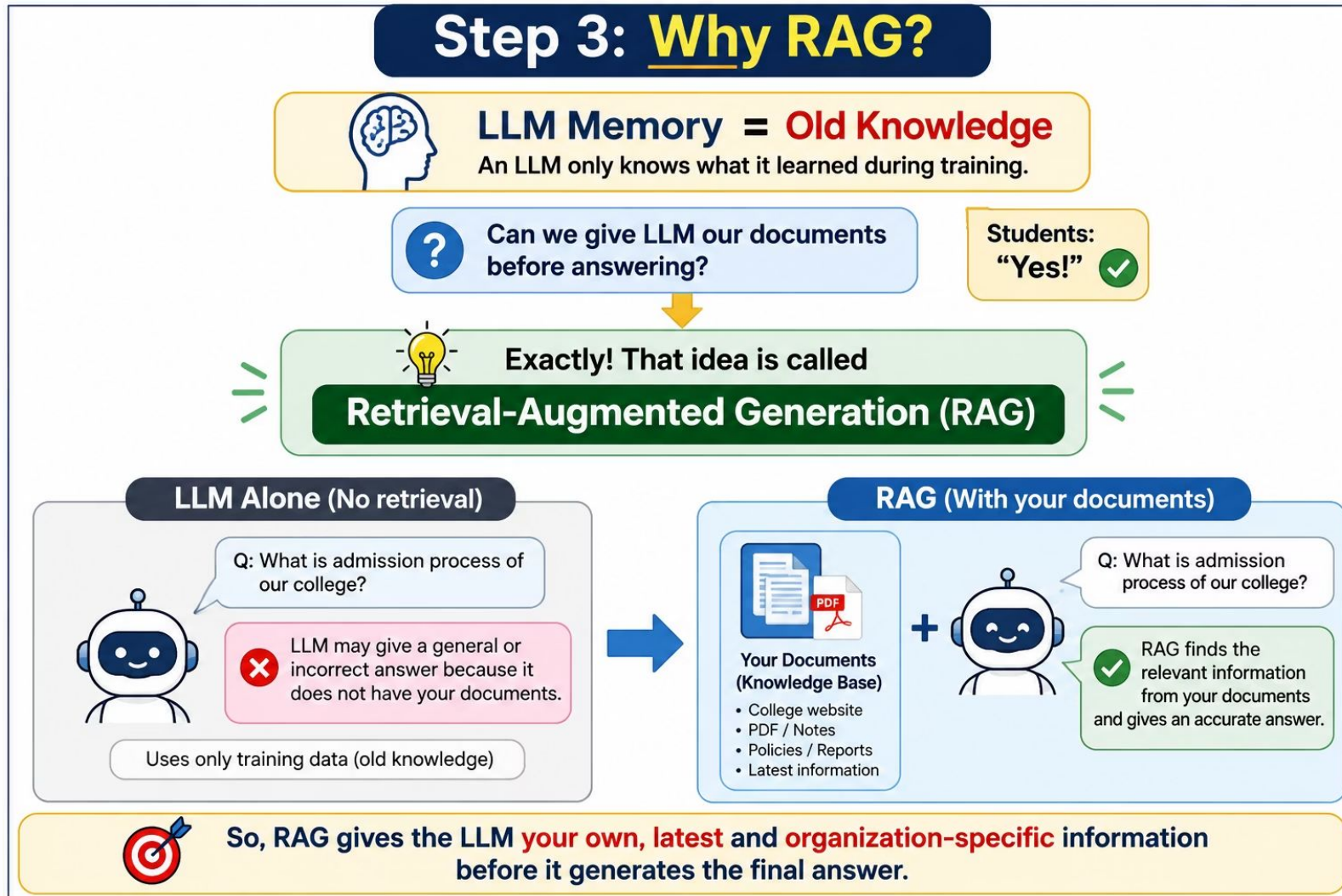
They don't automatically know



So, ChatGPT alone may give **wrong, outdated, or general** answers because it cannot access your **real-time or private information**.

Large Language Models like ChatGPT are trained on a fixed dataset with a knowledge cut-off date. They **do not automatically know** about today's news, your company's internal documents, your college's policies, or any private data you upload. This means they can produce **wrong, outdated, or overly generic answers** when queried about real-time or organisation-specific information – a critical limitation for practical deployment.

Why RAG? Bridging the Knowledge Gap

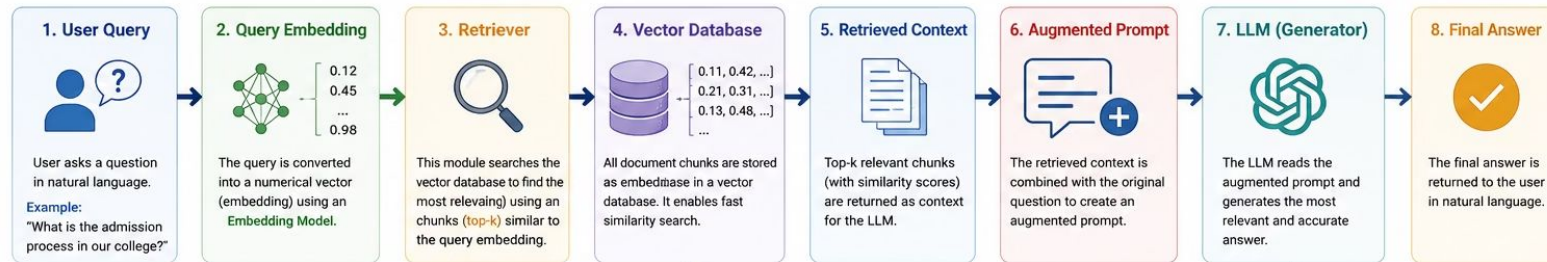


RAG solves the LLM knowledge limitation by supplying the model with **your own, latest, and organisation-specific documents** before it generates the final answer. Rather than retraining the entire model — which is expensive and time-consuming — RAG acts as a **dynamic memory extension**, feeding the LLM exactly what it needs at query time.

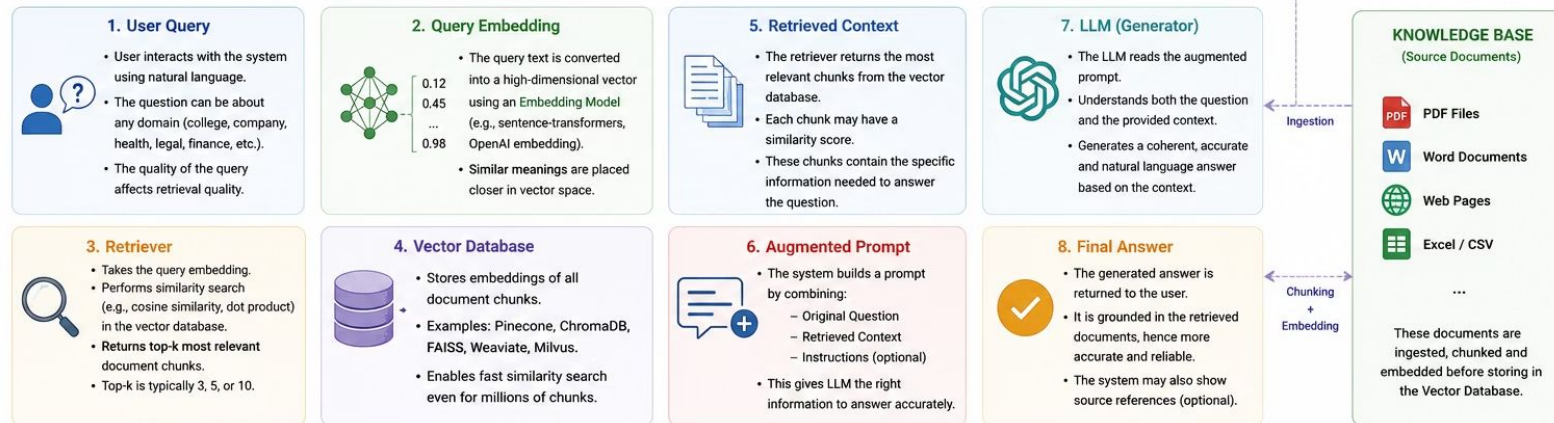
RAG Architecture — An Overview

RAG ARCHITECTURE (Retrieval-Augmented Generation)

RAG combines information retrieval from external knowledge sources with the generation power of LLMs to produce accurate, up-to-date and context-aware answers.



DETAILS OF EACH BLOCK



In short: RAG = **Retrieve** (find relevant information) + **Augment** (add it to the question) + **Generate** (LLM produces the answer)

The RAG pipeline combines **eight tightly integrated components**: User Query → Query Embedding → Retriever → Vector Database → Retrieved Context → Augmented Prompt → LLM Generator → Final Answer. Each stage is designed to ensure that the model receives precise, contextually relevant information, resulting in grounded and verifiable responses.

What is RAG?

Retrieval-Augmented Generation (RAG) is a hybrid AI paradigm that enhances language model responses by combining **information retrieval** with **text generation**. Instead of relying solely on knowledge encoded during training, a RAG system first **fetches relevant, up-to-date content** from external sources — such as documents, databases, or APIs — and then uses that content to generate a well-grounded answer.

Always Up-to-Date

Fetches live data; reduces hallucinations and outdated answers significantly.

Domain-Specific

Works exceptionally well with specialised content such as medical, legal, or academic data.

No Retraining Needed

New data can be added to the knowledge base without retraining the underlying LLM.

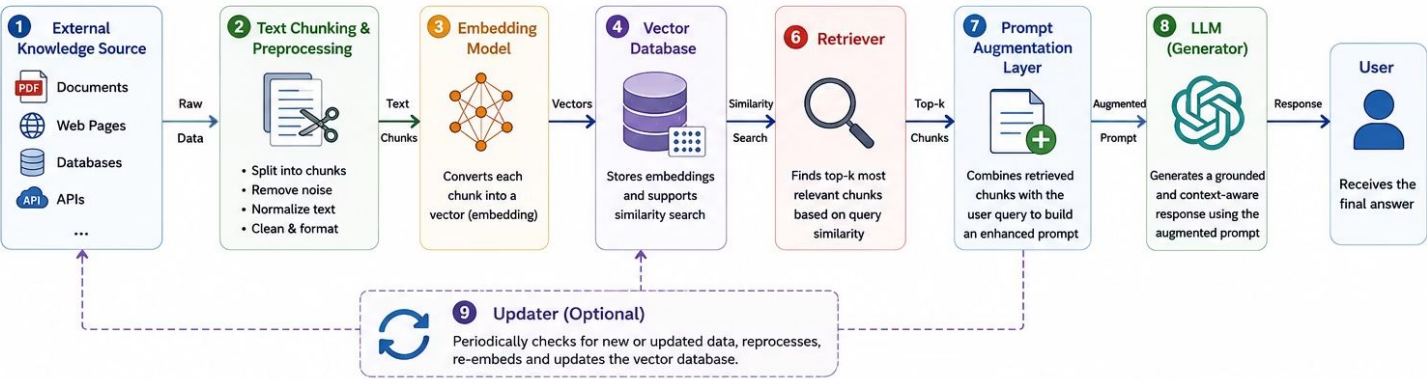
Personalised Responses

Can leverage user-specific or organisation-specific data for highly relevant answers.

Components of RAG

Components of RAG

RAG combines retrieval of relevant information with LLM generation to produce accurate, context-aware and up-to-date answers.



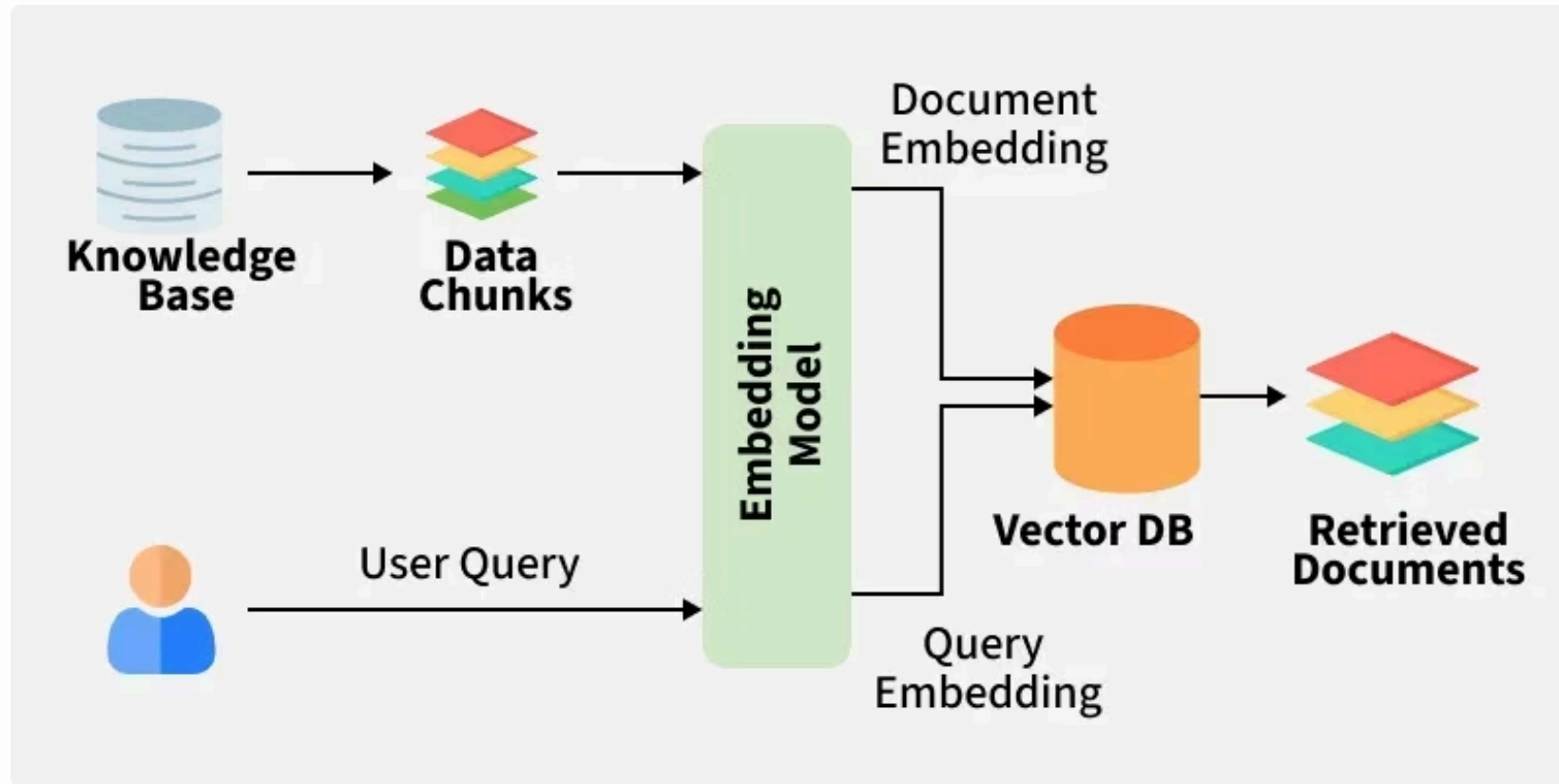
COMPONENTS EXPLAINED

1  External Knowledge Source Stores domain specific or general information like documents, APIs or databases.	6  Retriever Finds and returns the most relevant chunks from the database based on query similarity.
2  Text Chunking and Preprocessing Breaks large text into smaller, manageable chunks and cleans it for consistency.	7  Prompt Augmentation Layer Combines retrieved chunks with the user's query to provide context to the LLM.
3  Embedding Model Converts text into numerical vectors that capture semantic meaning.	8  LLM (Generator) Generates a grounded response using both the query and retrieved knowledge.
4  Vector Database Stores embeddings and enables similarity search for fast information retrieval.	9  Updater (Optional) Regularly refreshes and re-embeds data to keep the knowledge base up to date.
5  Query Encoder Transforms the user's query into a vector for comparison with stored embeddings.	

 **How it works:** The system retrieves relevant information from an external knowledge source, converts it into vectors, stores it in a vector database, and uses it to augment the user query. The LLM then generates accurate and contextually relevant answers.

RAG combines retrieval of relevant information with LLM generation to produce accurate, context-aware, and up-to-date answers. The pipeline spans from an **External Knowledge Source** through **Text Chunking**, **Embedding**, **Vector Storage**, **Retrieval**, and **Prompt Augmentation**, culminating in a grounded LLM response. An optional **Updater** module ensures the knowledge base remains current by periodically re-embedding new or changed documents.

Working of RAG



At its core, RAG operates on a **dual-stream embedding strategy**. Documents from the knowledge base are split into chunks, passed through an Embedding Model, and stored in a **Vector Database**. When a user submits a query, it is independently embedded and matched against stored document vectors. The most semantically similar chunks are then retrieved and forwarded – together with the original query – to the LLM for answer generation.

Working of RAG — Step by Step

01

External Data Ingestion

Documents from APIs, databases, and files are chunked, converted into vector embeddings, and stored in a vector database to build a searchable knowledge library.

02

Query Embedding & Retrieval

The user's query is converted into a vector and matched against stored embeddings to fetch the most semantically relevant data chunks.

03

Prompt Augmentation

Retrieved content is appended to the user's original query, giving the LLM rich contextual information it would otherwise not have access to.

04

Grounded Answer Generation

The LLM uses both the query and the retrieved context to produce a factually accurate, context-aware response with verifiable source grounding.

05

Continuous Data Refresh

External data and embeddings are refreshed in real-time or on a schedule, ensuring the system always retrieves the latest available information.

What is a Knowledge Base?

In the context of RAG, a **Knowledge Base** is a structured or unstructured collection of information that the system can search and retrieve from. It is the **external memory** that the LLM draws upon when answering queries. Virtually any form of organisational data can serve as a knowledge source.



Documents

PDFs, Word files, Excel spreadsheets, and scanned reports form the backbone of most enterprise knowledge bases.



Web Content

College websites, portals, and publicly accessible web pages can be crawled and indexed as knowledge sources.



Emails & Reports

Institutional emails, meeting minutes, annual reports, and policy circulars — everything becomes searchable knowledge.



Books & Manuals

Textbooks, laboratory manuals, syllabi, and reference books provide rich academic knowledge for student-facing chatbots.

Example: College Admission Chatbot

The Knowledge Base (College_Data/)

- Admission_Guidelines.pdf
- Fee_Structure.xlsx
- Hostel_Rules.docx
- AI_Club_Report.pdf
- Academic_Calendar.pdf
- Principal_Message.docx
- Library_Rules.pdf
- Placement_Statistics.xlsx
- www.sanjivani.edu.in
- NAAC_Report.pdf

Why This Matters

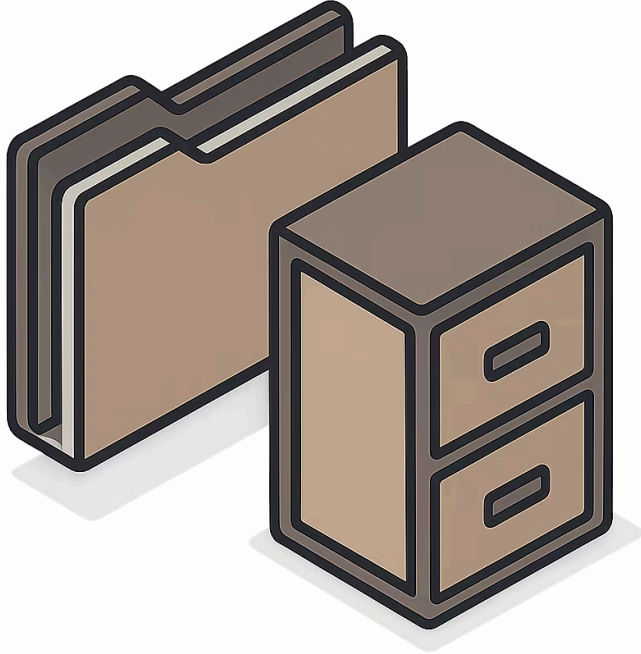
These ten files — spanning PDFs, spreadsheets, Word documents, and a live website — together form a comprehensive **Knowledge Base** for a college admission chatbot. Think of it as a **digital library** that the RAG system can search through in milliseconds.

When a student asks *"What is the fee for the AI programme?"*, the system retrieves only the relevant section from Fee_Structure.xlsx and generates a precise, current answer — without hallucinating figures.



Any collection of institutional files can become a powerful, queryable knowledge base with RAG.

Step 1 — Documents are Collected



The RAG system is pointed to a folder containing all relevant documents. At this initial stage, these are simply **ordinary files** — no processing has occurred yet. The system reads and catalogues every file in preparation for the ingestion pipeline.

Typical Knowledge Base Contents



Admission.pdf

Admission process guidelines and eligibility criteria.



Fee.xlsx

Programme-wise fee structure and payment schedules.



Hostel.docx

Hostel rules, room allocation, and warden contacts.



Library.pdf

Library timings, borrowing rules, and catalogue access.



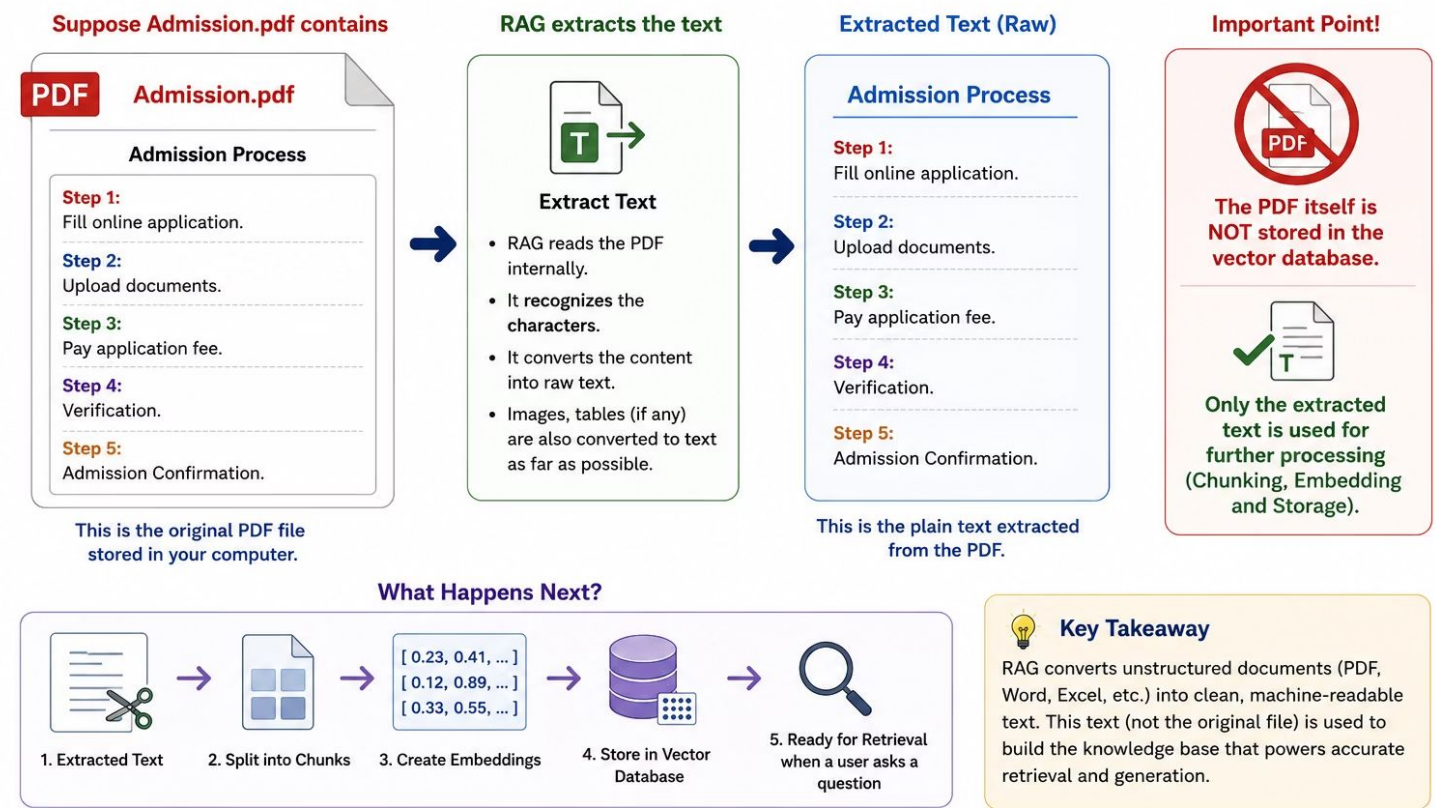
Website + Reports

Live web pages, NAAC reports, and placement statistics.

Step 2 — RAG Reads Every Document

STEP 2 RAG Reads Every Document

RAG first reads each document in the knowledge base and extracts the text content from it.

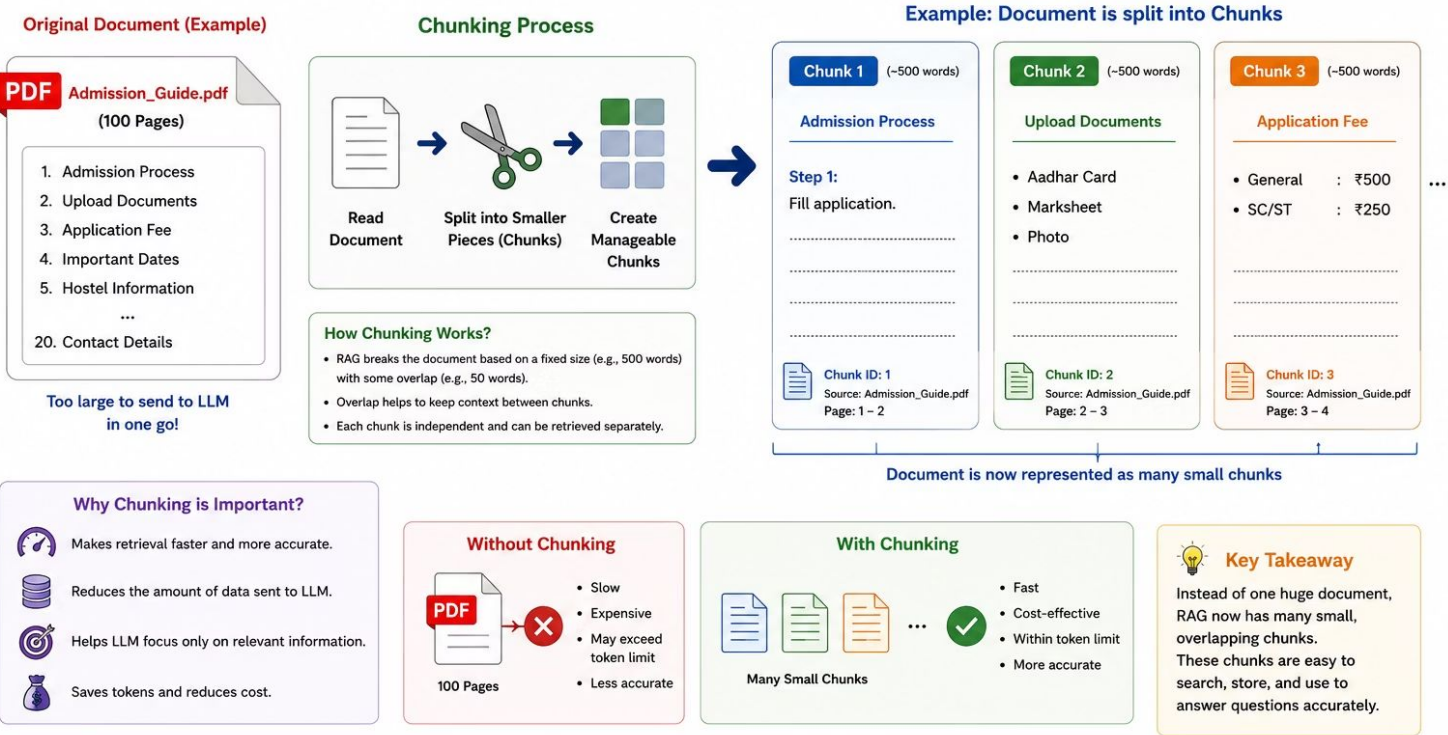


RAG reads each document in the knowledge base and **extracts the raw text content**. For a PDF like Admission.pdf, the system opens the file internally, recognises the characters, and converts all content — including text from tables and images where possible — into **plain machine-readable text**. Critically, the original PDF file itself is **not stored** in the vector database; only the extracted text moves forward for chunking, embedding, and storage.

Step 3 — Chunking: Splitting Large Documents

STEP 3 Chunking – Splitting Large Document into Small Pieces

💡 A 100-page PDF is too large for efficient retrieval and for the LLM to process. RAG splits it into smaller pieces called **chunks**.



Example If chunk size = 500 words and overlap = 50 words → A 100-page PDF (~50,000 words) → Will be converted into ~150 to 200 chunks

A 100-page PDF is too large to embed or send to an LLM in one go. RAG splits documents into **small, self-contained chunks** — typically 500 words each with a 50-word overlap to preserve context across boundaries. A 100-page document (~50,000 words) yields approximately **150 to 200 manageable chunks**. Chunking makes retrieval faster, more accurate, and token-efficient, ensuring the LLM focuses only on the most relevant information.

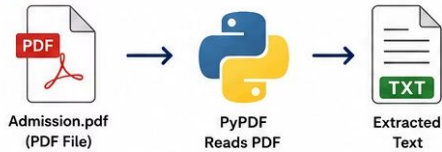
Reading PDF Files in Python Using PyPDF

READING PDF FILES IN PYTHON USING PyPDF



In RAG, the first step is to read PDF documents and extract their text.
Python does not understand PDF directly, so we **extract only the plain text**.

1 READ A SINGLE PDF



Python Code

```
1 from pypdf import PdfReader
2 reader = PdfReader("CollegeData/Admission.pdf")
3
4 text = ""
5 for page in reader.pages:
6     text += page.extract_text()
7
8 print(text)
```

Output

Admission Process
Step 1
Fill Online Application
Step 2
Upload Documents
Step 3
Pay Application Fee
Step 4
Verification
Step 5
Admission Confirmation



Notice

Python didn't keep the PDF.
It **extracted only the text**.



What Happened?

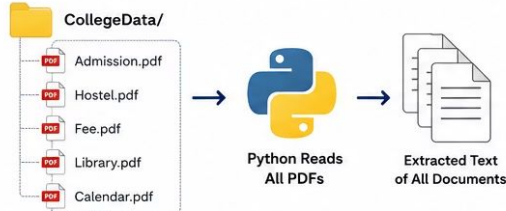
- PdfReader opens the PDF.
- page.extract_text() reads the text from each page.
- All text is stored in the variable text.
- The PDF file itself is **NOT** stored anywhere.



Key Takeaway

This is the first step in RAG:
Convert PDF → Plain Text

2 READ ALL PDFS IN A FOLDER



Python Code

```
1 import os
2 from pypdf import PdfReader
3 documents = []
4 folder = "CollegeData"
5
6 for file in os.listdir(folder):
7     if file.endswith(".pdf"):
8         reader = PdfReader(os.path.join(folder, file))
9         text = ""
10        for page in reader.pages:
11            text += page.extract_text()
12        documents.append(text)
13
14 print("Total Documents :", len(documents))
```

Output

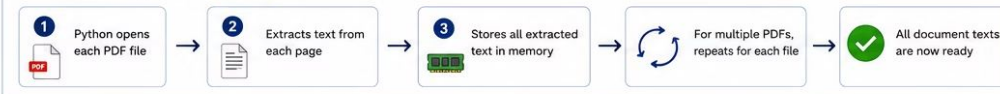
Total Documents : 5



What Next?

Now we have the text of all documents in the list documents.
These texts will be chunked in the next step.

How it Works (Summary Flow)

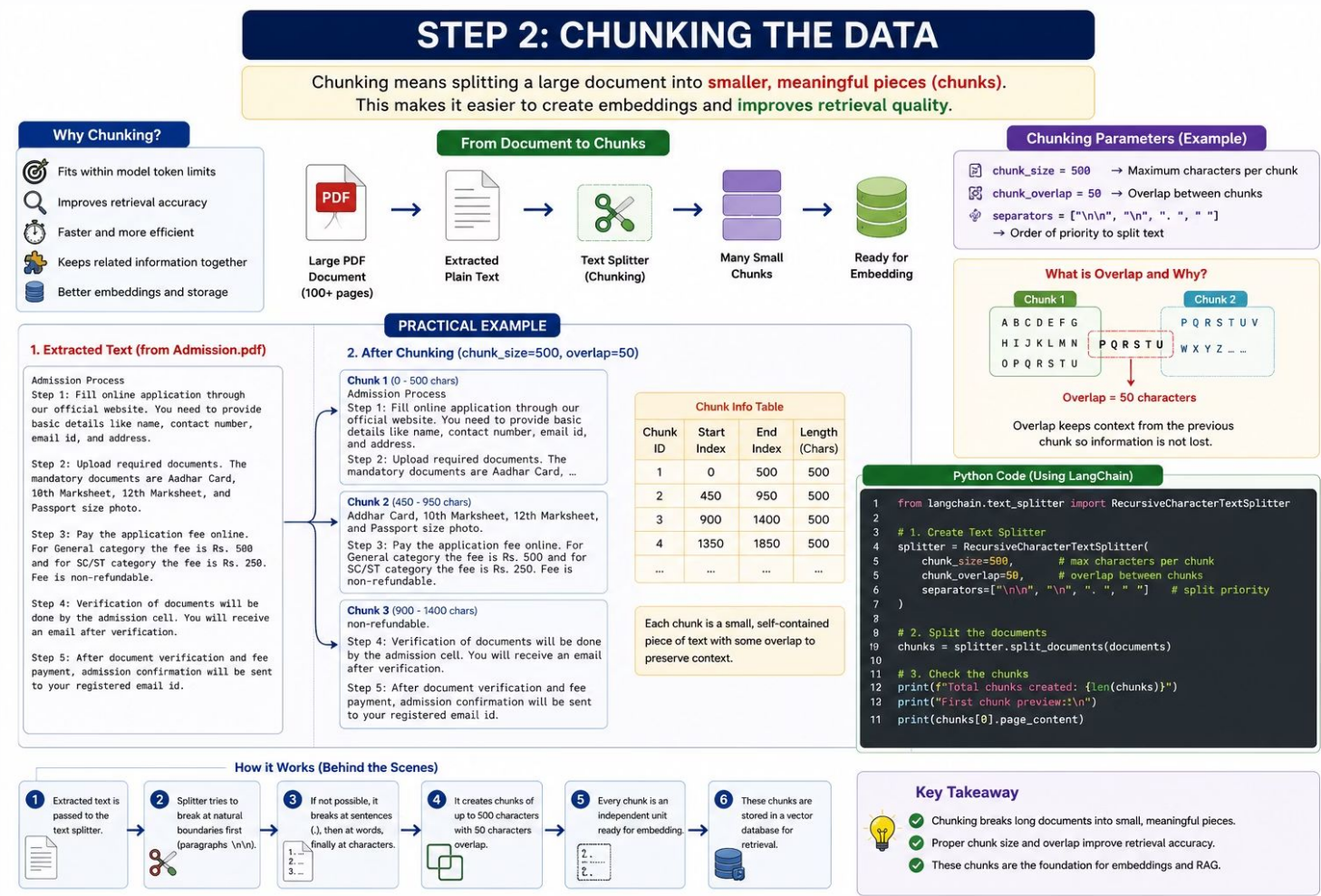


Result

All PDF content is now available as plain text for the next steps (chunking, embedding, storage...).

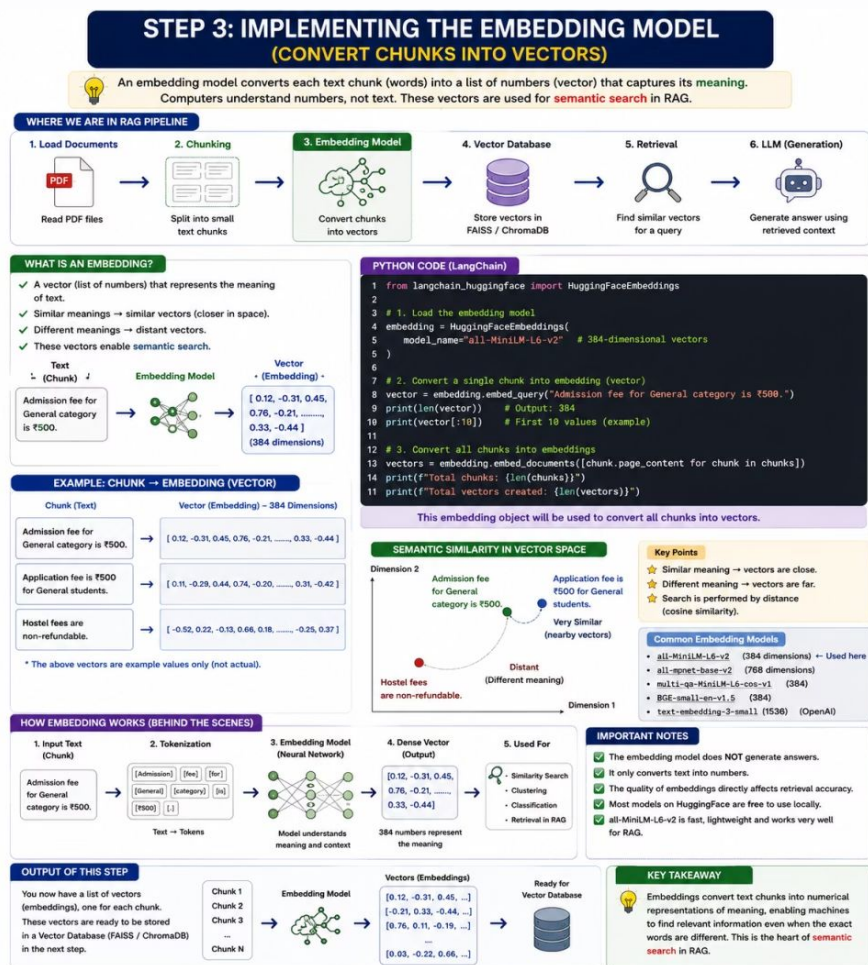
In RAG, the first step is to read PDF documents and extract their text content. Python does not understand PDFs directly, so we use the PyPDF library – specifically PdfReader – to open each file, iterate over its pages, and extract the text into a plain string. For a folder of PDFs, a simple loop processes all files and appends their content into a single list. **The PDF file itself is never stored** – only the extracted plain text moves forward to chunking and embedding.

Step 2 — Chunking the Data in Detail



Chunking means splitting a large document into **smaller, meaningful pieces** called chunks. This is the foundation of efficient RAG retrieval. Key parameters include `chunk_size` (maximum characters per chunk, e.g. 500) and `chunk_overlap` (characters shared between consecutive chunks, e.g. 50) to prevent loss of context at boundaries. In LangChain, `RecursiveCharacterTextSplitter` is the recommended splitter — it respects natural text boundaries and produces clean, overlapping chunks ready for embedding.

Embeddings — Converting Text to Vectors



What is an Embedding?

An **embedding** is a dense numerical vector that captures the **semantic meaning** of a piece of text. Words or sentences with similar meanings are mapped to vectors that are **close together** in high-dimensional space. This allows the RAG system to perform **semantic search** — finding the most relevant chunks by comparing vector distances — rather than relying on simple keyword matching.

Word Embeddings

Maps individual words to vectors (e.g., Word2Vec, GloVe).

Sentence Embeddings

Captures full sentence or paragraph meaning (e.g., Sentence-BERT, OpenAI Ada).

Similarity Search

Cosine similarity or dot product measures how close query and chunk vectors are.

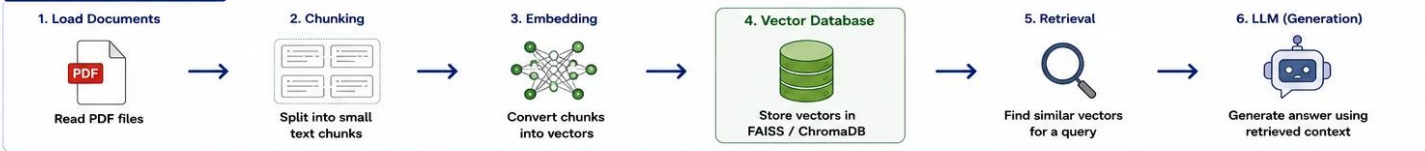
Vector Database — Storing and Searching Embeddings

STEP 4: IMPLEMENTING THE VECTOR DATABASE

Store and Search Embeddings Efficiently

A Vector Database stores **embeddings (vectors)** and allows **fast similarity search**.
It is the **memory** of your RAG system.

WHERE WE ARE IN RAG PIPELINE



WHAT IS A VECTOR DATABASE?

- ✓ Stores high-dimensional vectors (embeddings).
- ✓ Indexes vectors for ultra-fast similarity search.
- ✓ Returns the most relevant vectors (chunks) for a query.
- ✓ Examples: FAISS, ChromaDB, Pinecone, Weaviate, Milvus.

FEATURES

- 🔍 High Performance Search (millions of vectors in ms)
- 📈 Scalability (handle large volumes easily)
- 🔗 Similarity Search (cosine, dot product, L2, etc.)
- 🏷️ Metadata Filtering (filter by source, page, tags, etc.)
- 💾 Persistence (save and load the index)
- 🔄 Incremental Updates (add/delete vectors)

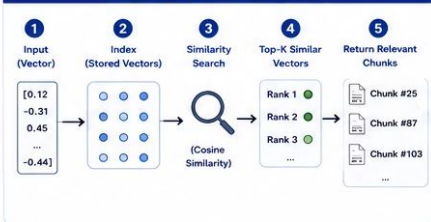
USES IN RAG

- 📄 Store embeddings of document chunks.
- 🔍 Given a user query, return top-k similar chunks.
- 🧠 Provide relevant context to the LLM.
- 🛡️ Improve accuracy and reduce hallucinations.

COMMON VECTOR DATABASES

Database	Best For
FAISS	Local use, high performance, large datasets
ChromaDB	Easy to use, local & cloud, great for RAG
Pinecone	Managed cloud service, production ready
Weaviate	Open source, schema & metadata rich
Milvus	Enterprise scale, distributed

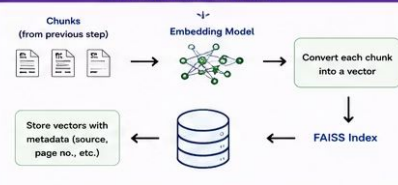
HOW VECTOR DATABASE WORKS



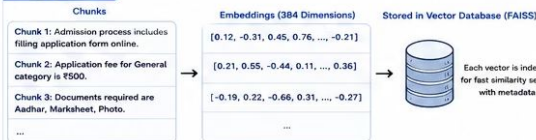
PYTHON CODE (Using LangChain + FAISS)

```
1 from langchain_community.vectorstores import FAISS
2 from langchain_huggingface import HuggingFaceEmbeddings
3
4 # 1. Load embedding model
5 embedding = HuggingFaceEmbeddings(model_name="all-MiniLM-L6-v2")
6
7 # 2. 'chunks' is the list of Document objects (from previous step)
8 # 'each chunk has .page_content and .metadata'
9
10 # 3. Create Vector Database (FAISS)
11 db = FAISS.from_documents(chunks, embedding)
12
13 # 4. Save the database locally
14 db.save_local("faiss_index")
15 print("Vector database created and saved!")
```

WHAT HAPPENS INTERNALLY?



EXAMPLE



SIMILARITY SEARCH EXAMPLE



KEY TAKEAWAY

- ✓ Vector DB stores embeddings, not raw text.
- ✓ Enables lightning-fast semantic search.
- ✓ Returns most relevant chunks for any query.
- ✓ FAISS is best for local development.
- ✓ ChromaDB/Pinecone are great for production.
- ✓ This step is the heart of efficient retrieval in RAG.

Always persist your vector database (save index).
Rebuilding embeddings for large docs is time consuming.

Common Similarity Metrics

Cosine Similarity – measures angle between vectors (most common)

Dot Product – measures magnitude and direction

L2 Distance – measures straight line distance between vectors

A **Vector Database** is a specialised data store optimised for storing, indexing, and querying high-dimensional embedding vectors at scale. Unlike traditional SQL databases that match exact values, vector databases perform **approximate nearest neighbour (ANN)** search – finding the chunks whose vectors are most similar to the query vector. Popular vector databases used in RAG systems include **FAISS**, **ChromaDB**, **Pinecone**, **Weaviate**, and **Qdrant**. They enable fast, scalable semantic search even over millions of document chunks.

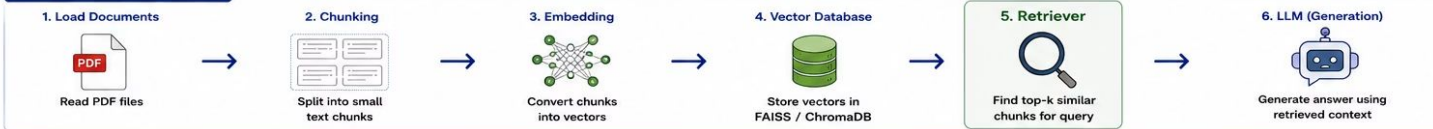
Retrieval — Finding the Most Relevant Chunks

STEP 5: RETRIEVER – FIND THE MOST RELEVANT CHUNKS



The Retriever takes a **user query**, converts it into a **vector (embedding)** and searches the Vector Database to find the **top-k most similar chunks**. These chunks are then used to provide context to the LLM.

WHERE WE ARE IN RAG PIPELINE



WHAT IS A RETRIEVER?

- ✓ Accepts a user query (natural language).
- ✓ Converts the query into an embedding.
- ✓ Searches the Vector Database for similar vectors.
- ✓ Returns top-k relevant document chunks.
- ✓ Acts as the memory search engine of RAG.

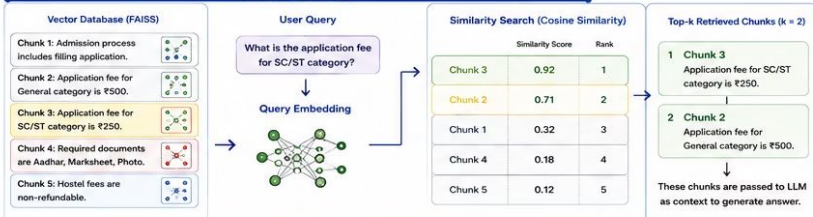
HOW RETRIEVER WORKS (IN DETAIL)

1. User asks a question (query).
2. Query is converted into an embedding using the same embedding model.
3. Embedding is compared with all stored vectors in the Vector DB.
4. Similarity score is computed (Cosine Similarity / L2 / Dot Product).
5. Top-k chunks with highest similarity are selected.
6. Retrieved chunks are passed to LLM as context.

PYTHON IMPLEMENTATION (LangChain + FAISS)

```
1 from langchain_community.vectorstores import FAISS
2 from langchain_huggingface import HuggingFaceEmbeddings
3
4 # 1. Load embedding model (same as used during indexing)
5 embedding = HuggingFaceEmbeddings(model_name="all-MiniLM-L6-v2")
6
7 # 2. Load existing FAISS vector database
8 db = FAISS.load_local("faiss_index", embedding, allow_dangerous_deserialization=True)
9
10 # 3. Create Retriever (searcher)
11 retriever = db.as_retriever(search_type="similarity", search_kwargs={"k": 3})
12
13 # 4. User query
14 query = "What is the application fee for SC/ST category?"
15
16 # 5. Retrieve top-k relevant chunks
17 docs = retriever.get_relevant_documents(query)
18
19 # 6. Print retrieved chunks
20 print(f"\nUser Query: {query}")
21 print(f"\nRetrieved Chunks:")
22 for i, doc in enumerate(docs):
23     print(f"\nChunk {i+1} (Score may vary):")
24     print(doc.page_content)
```

VISUAL EXAMPLE



ACTUAL OUTPUT (SAMPLE)

```
User Query: What is the application fee for SC/ST category?
Retrieved Chunks:

Chunk 1 (Score may vary):
Step 1: Pay the application fee online.
For SC/ST category the fee is Rs. 250. Fee is non-refundable.

Chunk 2 (Score may vary):
Step 3: Pay the application fee online.
For General category the fee is Rs. 500 and for SC/ST category the fee is Rs. 250.
```

KEY FEATURES OF RETRIEVER

- ✓ Fast similarity search over millions of vectors.
- ✓ Returns only the most relevant information.
- ✓ Improves accuracy and reduces hallucination.
- ✓ Configurable (top-k, search type, score threshold).
- ✓ Supports metadata filtering (source, page, date, etc.).

SEARCH TYPES IN LANGCHAIN

- similarity → Standard similarity search (default)
- mmr → Max Marginal Relevance (diverse results)
- similarity_score_threshold → Returns results above a score threshold

USES OF RETRIEVER

- Finds relevant information from large documents instantly.
- Provides precise context to LLM for better answers.
- Enables real-time Q&A over private data (RAG applications).
- Reduces incorrect answers by giving only relevant chunks.
- Works efficiently even with very large knowledge bases.

SUMMARY

The Retriever is the heart of RAG search. It takes a question, searches the Vector Database for the most relevant chunks using embeddings, and returns them to the LLM as context.

Similarity Metrics Used

- Cosine Similarity (Most Common)
- Dot Product (Magnitude + Direction)
- L2 Distance (Euclidean Distance)

Best Practices

- ✓ Use appropriate chunk size (e.g., 300–800 chars).
- ✓ Use overlap in chunking to preserve context.
- ✓ Use top-k = 3 to 10 (depends on use case).
- ✓ Filter by metadata to improve precision.

When a user submits a query, the **Retriever** module converts it into a vector using the same embedding model used during indexing. It then queries the vector database to return the **top-k most similar chunks** — typically the 3 to 5 chunks with the highest cosine similarity score. These retrieved chunks represent the most contextually relevant passages from the knowledge base and are passed forward to the **Prompt Augmentation Layer**, where they are combined with the original user query to form an enriched prompt for the LLM.

Prompt Augmentation & Answer Generation

STEP 6: PROMPT AUGMENTED LAYER – GIVE CONTEXT TO LLM



The Prompt Augmented Layer combines the user **query** with the **retrieved relevant chunks (context)** and sends it to the LLM. This helps the LLM generate accurate, **grounded** and hallucination-free answers.

WHERE WE ARE IN RAG PIPELINE



WHAT IS PROMPT AUGMENTATION?

- ✓ Takes the user question and the top retrieved chunks.
- ✓ Formats them into a well-structured prompt.
- ✓ Provides the LLM with the right context.
- ✓ Improves accuracy and reduces hallucinations.
- ✓ Also called Context Augmentation.

HOW PROMPT AUGMENTED LAYER WORKS

- 1 User asks a question.
- 2 Retriever returns top-k relevant chunks.
- 3 Chunks are inserted into a prompt template.
- 4 The augmented prompt is sent to the LLM.
- 5 LLM generates answer based on the given context.

TYPICAL PROMPT TEMPLATE

You are a helpful assistant for a college website.
Use the following context to answer the question.
If the answer is not in the context, say "I don't know".

Context:
{context}

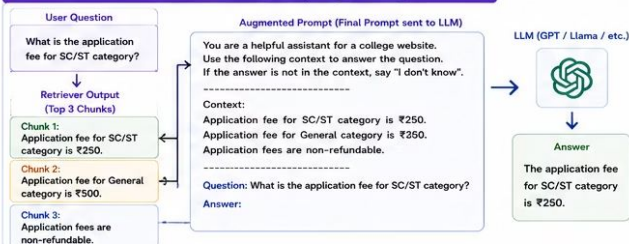
Question:
{question}

Answer:

PYTHON IMPLEMENTATION (LangChain)

```
1 from langchain.prompts import PromptTemplate
2 from langchain.chains import RetrievalQA
3
4 # 1. Define Prompt Template
5 template = """You are a helpful assistant for a college website.
6 Use the following context to answer the question.
7 If the answer is not in the context, say "I don't know".
8
9 Context:\n{context}\n\n
10 Question: {question}\n\n
11 Answer:"""
12
13 prompt = PromptTemplate(
14     input_variables=["context", "question"],
15     template=template
16 )
17
18 # 2. Create RetrievalQA chain (uses retriever + LLM)
19 qa = RetrievalQA.from_chain_type(
20     llm=llm, # your LLM (e.g., OpenAI, Groq, etc.)
21     chain_type="stuff", # stuff => put all chunks in prompt
22     retriever=retriever, # from previous step (FAISS retriever)
23     chain_type_kwargs={"prompt": prompt}
24 )
25
26 # 3. Ask a question
27 question = "What is the application fee for SC/ST category?"
28 result = qa.run(question)
29 print(result)
```

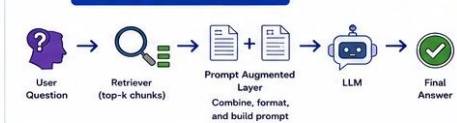
VISUAL EXAMPLE (Prompt Augmentation)



USES / BENEFITS



BEHIND THE SCENES FLOW



BEST PRACTICES

- ✓ Use a clear and strong system instruction.
- ✓ Include only top-k most relevant chunks.
- ✓ Limit total tokens in prompt (avoid very long context).
- ✓ Add metadata (source, page no.) if needed.
- ✓ Always tell the model: "If answer not in context, just say I don't know."



KEY TAKEAWAY

The Prompt Augmented Layer is the bridge between retrieval and generation. It gives the LLM the right context in the right format to produce accurate answers.



IMPORTANT TIP

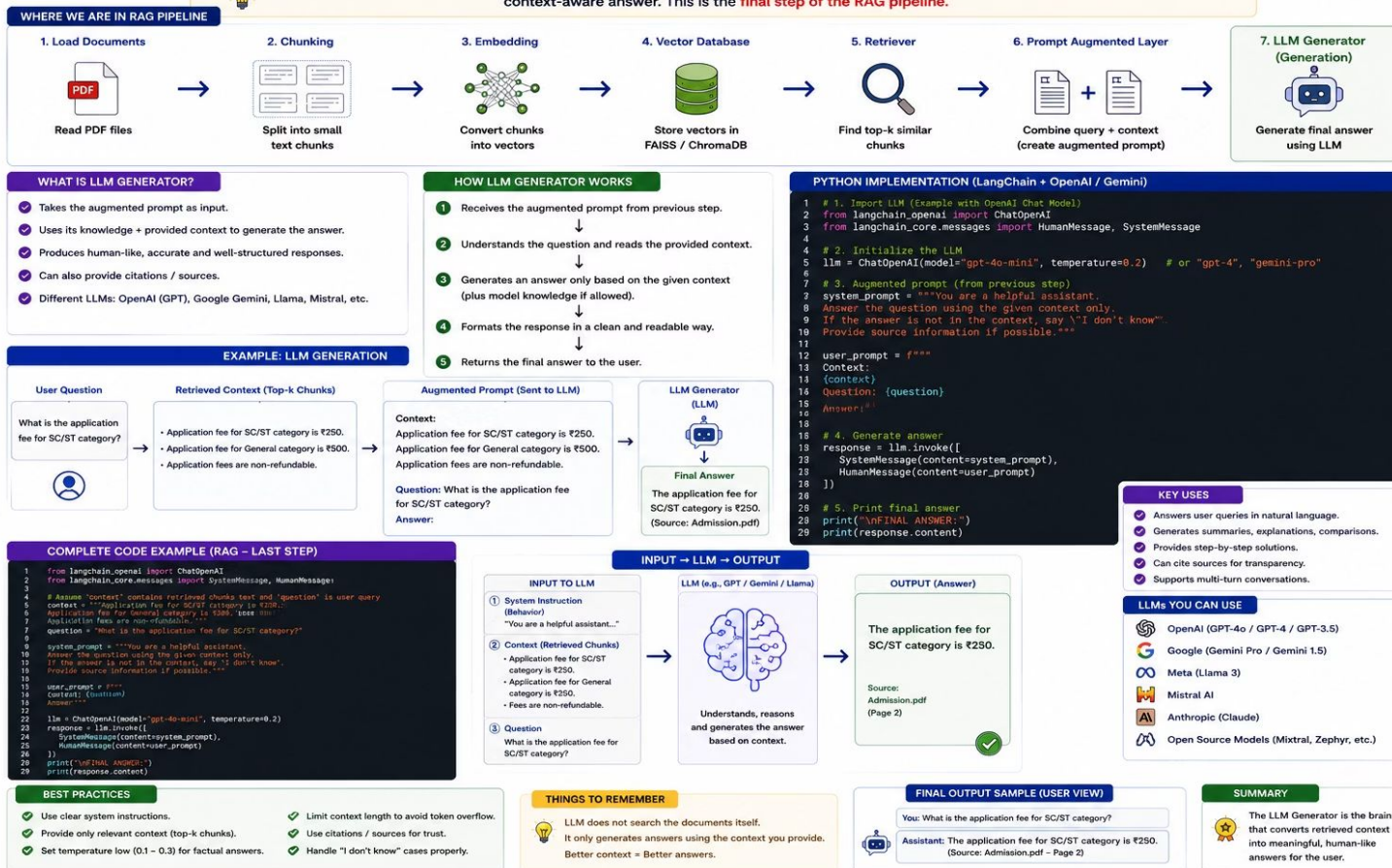
Good prompt + Good context (chunks) = High quality, trustworthy answers.

The **Augmented Prompt** is constructed by combining three elements: optional **system instructions** (e.g., "Answer only from the provided context"), the **retrieved chunks**, and the **original user query**. This enriched prompt is sent to the LLM, which then generates a **grounded, context-aware response**. Because the answer is derived from retrieved documents rather than memorised training data, it is significantly more accurate, verifiable, and up-to-date – greatly reducing the risk of hallucination.

End-to-End RAG Pipeline — Putting It All Together

STEP 7: LLM GENERATOR – GENERATE THE FINAL ANSWER

The LLM Generator takes the augmented prompt (question + retrieved context) and generates a natural, accurate and context-aware answer. This is the **final step of the RAG pipeline**.



The complete RAG pipeline flows from **document ingestion** → **text extraction** → **chunking** → **embedding** → **vector storage** → **semantic retrieval** → **prompt augmentation** → **LLM generation** → **final answer**. Each stage is purpose-built to ensure the model receives precise, grounded, and contextually rich information. RAG represents a paradigm shift: rather than encoding all knowledge into model weights, it externalises knowledge into a living, updatable knowledge base — making AI systems more **accurate, transparent, maintainable, and trustworthy** for real-world deployment.

❏ RAG = Retrieve (find relevant information) + Augment (add it to the prompt) + Generate (LLM produces the answer).